UNIT 1 45 min

Your First Python Program

Print output, store values in variables, write comments, and read error messages.

By the end of this unit you will be able to

- Run a Python statement and explain what `print()` does
- Create variables with the assignment operator `=`
- Follow the naming rules for variables and choose readable names
- Write comments that explain intent
- Read a simple error message and find the line that caused it

Why Python

Python reads close to plain English, which makes it a good first language and a strong tool for working professionals. The same language runs web sites, data analysis, automation, and machine-learning models. Everything you learn in this course applies to all of those.

Your first program

A Python program is a list of **statements**. Python runs them from top to bottom, one at a time. The `print()` function displays a value on the screen (the output).

hello.py

```
print("Hello, world!")
print("Python is running.")
```

> **Tip**
> Press **Run** ▶ on any code box to execute it in your browser, or copy the code and paste it into the Sandbox page.

Variables store values

A **variable** is a name that points to a value. You create one with the assignment operator `=`. The right side is evaluated first, then the result is stored under the name on the left. Read `=` as "gets", not as "equals".

variables.py

```
name = "Ada"
age = 36
print(name)
print(age)
```

Trace: a variable is reassigned TRACE

```
1  x = 5
2  y = x + 2
3  x = 10
4  print(y)
```

Step	Line run	Variables after the line	Output so far
1	1	`x = 5`	`—`
2	2	`x = 5, y = 7`	`—`
3	3	`x = 10, y = 7`	`—`
4	4	`x = 10, y = 7`	`7`

Naming rules

- Names may use letters, digits, and underscores `_`.
- A name cannot start with a digit. `2nd_place` is illegal; `second_place` is fine.
- Names are **case-sensitive**. `Score` and `score` are two different variables.
- Names cannot be Python keywords such as `for`, `if`, `while`, or `True`.
- Use **snake_case** (lowercase words joined by underscores). It is the Python convention.

Name	Legal?	Why
`total_price`	Yes	Descriptive snake_case
`x1`	Yes	Letters then a digit is fine
`1st`	No	Starts with a digit
`my-name`	No	A hyphen is read as minus
`class`	No	Reserved keyword

Comments

A comment starts with `#`. Python ignores everything after it on that line. Use comments to explain *why* code does something, not to repeat what the code already says.

commented.py

```
# Store the sales tax rate so it is easy to change later.
TAX_RATE = 0.07

price = 40          # the price before tax
print(price * (1 + TAX_RATE))
```

Reading errors

When Python cannot run a line, it stops and prints an error. The last line says what went wrong, and the line above it says where. Errors are information, not failure.

broken.py

```
print(score)
```

That produces `NameError: name 'score' is not defined`. The name `score` was never assigned. Fix it by creating the variable first.

Predict the output THINK FIRST

Before you run anything, decide what each program prints.

```
print(2 + 3)
```

Answer: `5`

```
print("2 + 3")
```

Answer: `2 + 3`
Inside quotes, Python treats the text literally. It is a string, not math.

```
a = 1
b = a
a = 2
print(b)
```

Answer: `1`
`b` received the value of `a` at that moment. Later changes to `a` do not affect `b`.

```
print(score)
```

Answer: `NameError`
The name was never assigned, so Python cannot find it.

Practice

▼ **Exercise 1 — Introduce yourself**

Create three variables: `name` (text), `favorite_number` (a whole number), and `city` (text). Print each one on its own line.

▼ **Hint**

Text goes in quotes. Numbers do not.

▼ **Show solution**

Solution

```
name = "Jordan"
favorite_number = 7
city = "Cumming"
print(name)
print(favorite_number)
print(city)
```

▼ **Exercise 2 — Swap two values**

Given `a = 1` and `b = 2`, write code that swaps them so `a` ends with 2 and `b` ends with 1. Use a third variable called `temp`.

▼ **Hint**

Save one value before you overwrite it.

▼ **Show solution**

Solution

```
a = 1
b = 2
temp = a
a = b
b = temp
print(a, b)
```

Discussion

1. Why do you think Python uses the `=` symbol for assignment when math uses it for equality? What does that cost a beginner?
2. Give an example of a comment that repeats the code and one that explains why. Which one is more useful six months from now?
3. Which naming rule do you think causes the most confusion for new programmers, and why?

▼ **Facilitator notes**

Students often read `x = x + 1` as an impossible equation. Have them trace it with the Trace tool above, or ask: "what is `x` before the line? after?"

Key terms

Statement

One complete instruction the interpreter runs, such as `print("hi")` or `x = 3`.

Interpreter

The program that reads Python code and runs it. Python code runs line by line through the interpreter.

Variable

A name that refers to a value stored in memory. The value can be changed by assigning a new one.

Assignment

Storing a value under a name using `=`. The right side is computed first.

print()

A built-in function that writes its arguments to the output, separated by spaces, followed by a newline.

Comment

Text after `#` that Python ignores. Used to explain intent to human readers.

Keyword

A word reserved by the language (for example `if`, `for`, `True`). It cannot be used as a variable name.

snake_case

Naming style with lowercase words joined by underscores, such as `total_price`. Python's convention for variables and functions.

NameError

The error raised when code uses a name that has not been assigned.

SyntaxError

The error raised when code breaks the grammar of the language, such as a missing quotation mark or parenthesis.

See all terms in the glossary →